

In computer science, **merge sort** (also commonly spelled as **mergesort** and as **merge-sort**^[2]) is an efficient, general-purpose, and comparison-based sorting algorithm. Most implementations of merge sort are stable, which means that the relative order of equal elements is the same between the input and output. Merge sort is a divide-and-conquer algorithm that was invented by John von Neumann in 1945.^[3] A detailed description and analysis of bottom-up merge sort appeared in a report by Goldstine and von Neumann as early as 1948.^[4]

Algorithm

Conceptually, a merge sort works as follows:

1. Divide the unsorted list into n sub-lists, each containing one element (a list of one element is considered sorted).
2. Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining. This will be the sorted list.

Top-down implementation

Example C-like code using indices for top-down merge sort algorithm that recursively splits the list (called *runs* in this example) into sublists until sublist size is 1, then merges those sublists to produce a sorted list. The copy back step is avoided with alternating the direction of the merge with each level of recursion (except for an initial one-time copy, that can be avoided too).

As a simple example, consider an array with two elements. The elements are copied to $B[]$, then merged back to $A[]$. If there are four elements, when the bottom of the recursion level is reached, single element runs from $A[]$ are merged to $B[]$, and then at the next higher level of recursion, those two-element runs are merged to $A[]$. This pattern continues with each level of recursion.

```
// Array A[] has the items to sort; array B[] is a work array.
void TopDownMergeSort(A[], B[], n)
{
    CopyArray(A, 0, n, B);           // one time copy of A[] to B[]
    TopDownSplitMerge(A, 0, n, B);   // sort data from B[] into A[]
}

// Split A[] into 2 runs, sort both runs into B[], merge both runs from B[] to A[]
// iBegin is inclusive; iEnd is exclusive (A[iEnd] is not in the set).
void TopDownSplitMerge(B[], iBegin, iEnd, A[])
{
    if (iEnd - iBegin <= 1)           // if run size == 1
        return;                     // consider it sorted
    // split the run longer than 1 item into halves
    iMiddle = (iEnd + iBegin) / 2;    // iMiddle = mid point
    // recursively sort both runs from array A[] into B[]
    TopDownSplitMerge(A, iBegin, iMiddle, B); // sort the left run
    TopDownSplitMerge(A, iMiddle, iEnd, B);  // sort the right run
    // merge the resulting runs from array B[] into A[]
    TopDownMerge(B, iBegin, iMiddle, iEnd, A);
}

// Left source half is A[ iBegin:iMiddle-1].
// Right source half is A[iMiddle:iEnd-1 ].
// Result is           B[ iBegin:iEnd-1 ].
void TopDownMerge(B[], iBegin, iMiddle, iEnd, A[])
{
    i = iBegin, j = iMiddle;
```

```

// While there are elements in the left or right runs...
for (k = iBegin; k < iEnd; k++) {
    // If left run head exists and is <= existing right run head.
    if (i < iMiddle && (j >= iEnd || A[i] <= A[j])) {
        B[k] = A[i];
        i = i + 1;
    } else {
        B[k] = A[j];
        j = j + 1;
    }
}
}

void CopyArray(A[], iBegin, iEnd, B[])
{
    for (k = iBegin; k < iEnd; k++)
        B[k] = A[k];
}

```

Sorting the entire array is accomplished by `TopDownMergeSort(A, B, length(A))`.

Bottom-up implementation

Example C-like code using indices for bottom-up merge sort algorithm which treats the list as an array of n sublists (called *runs* in this example) of size 1, and iteratively merges sub-lists back and forth between two buffers:

```

// array A[] has the items to sort; array B[] is a work array
void BottomUpMergeSort(A[], B[], n)
{
    // Each 1-element run in A is already "sorted".
    // Make successively longer sorted runs of length 2, 4, 8, 16... until the whole array is sorted.
    for (width = 1; width < n; width = 2 * width)
    {
        // Array A is full of runs of length width.
        for (i = 0; i < n; i = i + 2 * width)
        {
            // Merge two runs: A[i:i+width-1] and A[i+width:i+2*width-1] to B[]
            // or copy A[i:n-1] to B[] ( if (i+width >= n) )
            BottomUpMerge(A, i, min(i+width, n), min(i+2*width, n), B);
        }
        // Now work array B is full of runs of length 2*width.
        // Copy array B to array A for the next iteration.
        // A more efficient implementation would swap the roles of A and B.
        CopyArray(B, A, n);
        // Now array A is full of runs of length 2*width.
    }
}

// Left run is A[iLeft :iRight-1].
// Right run is A[iRight:iEnd-1 ].
void BottomUpMerge(A[], iLeft, iRight, iEnd, B[])
{
    i = iLeft, j = iRight;
    // While there are elements in the left or right runs...
    for (k = iLeft; k < iEnd; k++) {
        // If left run head exists and is <= existing right run head.
        if (i < iRight && (j >= iEnd || A[i] <= A[j])) {
            B[k] = A[i];
            i = i + 1;
        } else {
            B[k] = A[j];
            j = j + 1;
        }
    }
}

void CopyArray(B[], A[], n)
{

```

```

    for (i = 0; i < n; i++)
        A[i] = B[i];
}

```

Top-down implementation using lists

Pseudocode for top-down merge sort algorithm which recursively divides the input list into smaller sublists until the sublists are trivially sorted, and then merges the sublists while returning up the call chain.

```

function merge_sort(list m) is
    // Base case. A list of zero or one elements is sorted, by definition.
    if length of m ≤ 1 then
        return m

    // Recursive case. First, divide the list into equal-sized sublists
    // consisting of the first half and second half of the list.
    // This assumes lists start at index 0.
    var left := empty list
    var right := empty list
    for each x with index i in m do
        if i < (length of m)/2 then
            add x to left
        else
            add x to right

    // Recursively sort both sublists.
    left := merge_sort(left)
    right := merge_sort(right)

    // Then merge the now-sorted sublists.
    return merge(left, right)

```

In this example, the **merge** function merges the left and right sublists.

```

function merge(left, right) is
    var result := empty list

    while left is not empty and right is not empty do
        if first(left) ≤ first(right) then
            append first(left) to result
            left := rest(left)
        else
            append first(right) to result
            right := rest(right)

    // Either left or right may have elements left; consume them.
    // (Only one of the following loops will actually be entered.)
    while left is not empty do
        append first(left) to result
        left := rest(left)
    while right is not empty do
        append first(right) to result
        right := rest(right)
    return result

```

Bottom-up implementation using lists

Pseudocode for bottom-up merge sort algorithm which uses a small fixed size array of references to nodes, where **array[i]** is either a reference to a list of size 2^i or nil. *node* is a reference or pointer to a node. The **merge()** function would be similar to the one shown in the top-down merge lists example, it merges two already sorted lists, and handles empty lists. In this case, **merge()** would use *node* for its input parameters and return value.

```

function merge_sort(node head) is
    // return if empty list
    if head = nil then
        return nil
    var node array[32]; initially all nil
    var node result
    var node next
    var int i
    result := head
    // merge nodes into array
    while result ≠ nil do
        next := result.next;
        result.next := nil
        for (i = 0; (i < 32) && (array[i] ≠ nil); i += 1) do
            result := merge(array[i], result)
            array[i] := nil
        // do not go past end of array
        if i = 32 then
            i -= 1
            array[i] := result
            result := next
    // merge array into single list
    result := nil
    for (i = 0; i < 32; i += 1) do
        result := merge(array[i], result)
    return result

```

Top-down implementation in a declarative style

Haskell-like pseudocode, showing how merge sort can be implemented in such a language using constructs and ideas from functional programming.

```

mergeSort :: Ord a => [a] -> [a]
mergeSort [] = []
mergeSort [x] = [x]
mergeSort xs = merge (mergeSort l, mergeSort r)
    where (l, r) = splitAt (length xs `div` 2) xs

merge :: Ord a => ([a], [a]) -> [a]
merge ([], xs) = xs
merge (xs, []) = xs
merge (x:xs, y:ys) | x <= y = x : merge(xs, y:ys)
                  | otherwise = y : merge(x:xs, ys)

```

Analysis

In sorting n objects, merge sort has an average and worst-case performance of $O(n \log n)$ comparisons. If the running time (number of comparisons) of merge sort for a list of length n is $T(n)$, then the recurrence relation $T(n) = 2T(n/2) + n$ follows from the definition of the algorithm (apply the algorithm to two lists of half the size of the original list, and add the n steps taken to merge the resulting two lists).^[5] The closed form follows from the master theorem for divide-and-conquer recurrences.

The number of comparisons made by merge sort in the worst case is given by the sorting numbers. These numbers are equal to or slightly smaller than $(n \lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1)$, which is between $(n \lg n - n + 1)$ and $(n \lg n + n + O(\lg n))$.^[6] Merge sort's best case takes about half as many iterations as its worst case.^[7]

For large n and a randomly ordered input list, merge sort's expected (average) number of comparisons

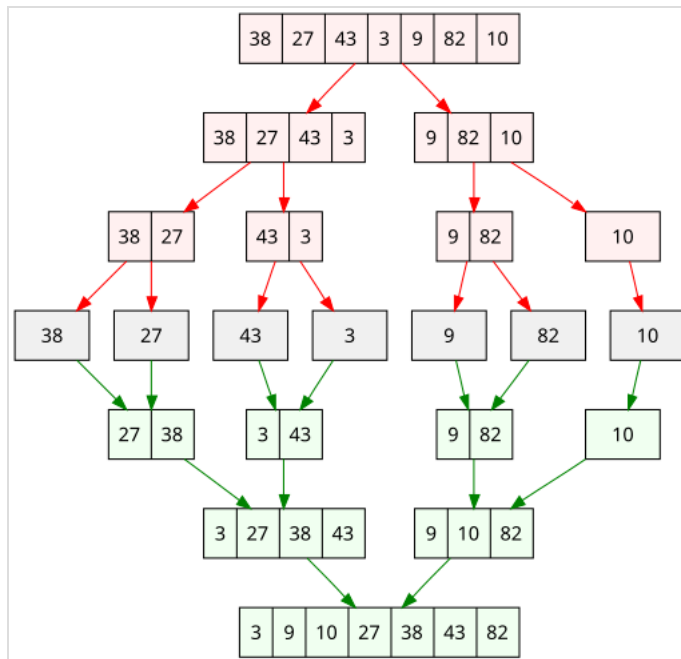
approaches $\alpha \cdot n$ fewer than the worst case,

where $\alpha = -1 + \sum_{k=0}^{\infty} \frac{1}{2^k + 1} \approx 0.2645$.

In the *worst* case, merge sort uses approximately 39% fewer comparisons than quicksort does in its *average* case, and in terms of moves, merge sort's worst case complexity is $O(n \log n)$ - the same complexity as quicksort's best case.^[7]

Merge sort is more efficient than quicksort for some types of lists if the data to be sorted can only be efficiently accessed sequentially, and is thus popular in languages such as Lisp, where sequentially accessed data structures are very common. Unlike some (efficient) implementations of quicksort, merge sort is a stable sort.

Merge sort's most common implementation does not sort in place;^[8] therefore, the memory size of the input must be allocated for the sorted output to be stored in (see below for variations that need only $n/2$ extra spaces).



A recursive merge sort algorithm used to sort an array of 7 integer values. These are the steps a human would take to emulate merge sort (top-down).